

University of Victoria
Faculty of Engineering
Summer 2026 ENGR 446 Final Report

**Design and Analysis of a Pipelined RTL Hardware Accelerator
for Real-Time Sobel Edge Detection**

Robin Zerr
V01013510
ENGR 446
Electrical Engineering
Email: zerrobin@gmail.com

July 3, 2026

In partial fulfillment of the requirements of the B.Eng. Degree

Monty Raisinghani, Instructor
ENGR 446
Faculty of Engineering
University of Victoria
Victoria, BC

July 3, 2026

Re: Design and Analysis of a Pipelined RTL Hardware Accelerator for Real-Time Sobel Edge Detection

Dear Mr. Raisinghani,

Attached is the technical report “Design and Analysis of a Pipelined RTL Hardware Accelerator for Real-Time Sobel Edge Detection,” submitted in partial fulfillment of the requirements of ENGR 446 in the electrical engineering program.

I am a fourth-year electrical engineering student, and this project draws on my coursework and co-op experience in digital systems design. The report examines whether a dedicated hardware accelerator can guarantee real-time Sobel edge detection on a streaming video source, something a general-purpose processor cannot promise. It compares a sequential and a pipelined register-transfer-level architecture, selects the pipelined design through a weighted assessment, and analyses its throughput, latency, arithmetic, and resource usage. The design was implemented in VHDL and verified in simulation against a software reference model; synthesis results are presented as estimates where noted, and the analysis is limited to grayscale images and the standard 3×3 Sobel operator.

The principal recommendation is to proceed with the pipelined architecture, which sustained one output pixel per clock cycle in simulation with a perfect match against the reference model, providing roughly ten times the required real-time pixel rate at a 100 MHz clock.

Thank you for taking the time to review this report. I look forward to your feedback.

Sincerely,

Robin Zerr
V01013510

Table of Contents

Executive Summary.....	iii
Glossary.....	iv
List of Symbols.....	v
1.0 Introduction.....	1
2.0 Background.....	1
3.0 Objective.....	2
3.1 Problem.....	2
3.2 Purpose.....	2
3.3 Aims.....	2
3.4 Limitations.....	2
4.0 Potential Solutions.....	3
4.1 Sequential RTL Filter.....	3
4.2 Pipelined RTL Filter.....	3
5.0 Initial Assessment.....	3
6.0 Proposed Solution.....	4
7.0 Engineering Analysis.....	6
7.1 Real-Time Throughput Requirement.....	6
7.2 Sequential RTL Throughput.....	6
7.3 Pipelined RTL Throughput.....	7
7.4 Latency Analysis.....	7
7.5 Sobel Arithmetic Analysis.....	7
7.6 Resource Usage Analysis.....	8
7.7 Verification Results.....	9
7.8 Sensitivity and Limitations of the Analysis.....	10
8.0 Discussion.....	11
9.0 Conclusions.....	11
10.0 Recommendations.....	12
References.....	13
Appendix A: Supporting Calculations.....	A-1
Appendix B: Simulation Waveforms and Synthesis Reports.....	B-1

List of Figures

Figure 1: Block diagram of the pipelined Sobel accelerator.....	5
Figure 2: The 3×3 pixel window and the Sobel kernels.....	5

Figure 3: Pipeline timing..... 5

Figure 4: Verification of the 640×480 run..... 10

List of Tables

Table 1: Weighted decision matrix for selecting the RTL Sobel architecture.....3

Table 2: Expected synthesis results (pre-synthesis estimates).....8

Table 3: Verification results, RTL versus golden model.....9

Executive Summary

Real-time embedded vision systems must process every pixel of a continuous video stream within a fixed time budget. General-purpose processors execute image filtering as a sequence of instructions whose timing varies with memory and cache behaviour, so they cannot guarantee the throughput and bounded latency that sustained operation requires. This report determines the most suitable register-transfer-level (RTL) hardware architecture for a real-time Sobel edge-detection accelerator operating on a streaming grayscale image source.

Two candidate architectures were compared using a weighted decision matrix: a sequential filter that shares arithmetic hardware over several clock cycles per pixel, and a pipelined filter that uses line buffers and shift registers to produce one output pixel per clock cycle. The pipelined design was selected, implemented in VHDL, and analysed for throughput, latency, arithmetic behaviour, and resource usage. The design was then verified in simulation against a software reference model across four image sizes up to a full 640×480 frame.

The application requires a sustained rate of 9.216 million pixels per second for 640×480 video at 30 frames per second. In simulation the pipelined design sustained one output pixel per clock cycle, or 100 million pixels per second at a 100 MHz clock, roughly ten times the requirement. Every output pixel matched the reference model exactly in all four verification runs, and the measured pipeline latency of 1,285 clock cycles (12.85 μ s) agreed exactly with the analytical prediction. The arithmetic uses no multipliers, so the hardware cost is dominated by two small row buffers of on-chip memory; synthesis resource figures remain estimates until the Vivado reports are complete.

The pipelined RTL filter is therefore recommended. Its throughput margin already covers full-HD video at 30 frames per second, and the line-buffer streaming pattern generalises to other convolution kernels, making the accelerator a reusable building block for embedded vision. Recommended next steps are to complete Vivado synthesis and timing verification, add a standard streaming interface, and extend the design toward colour processing and configurable kernels.

Glossary

Term	Definition
ASIC	Application-specific integrated circuit; a custom-fabricated chip designed for a particular function.
AXI-Stream	A streaming data interface standard using a ready/valid handshake to transfer data with back-pressure.
BRAM	Block RAM; dedicated on-chip memory blocks within an FPGA.
Convolution (2D)	An operation in which a kernel is slid across an image and combined with local pixels by multiply-accumulate.
CPU	Central processing unit; a general-purpose instruction-executing processor.
DSP block	Digital signal processing block; a hardened FPGA multiply-accumulate resource.
Edge detection	Identification of intensity boundaries in an image, typically via gradient estimation.
FPGA	Field-programmable gate array; a reconfigurable logic device.
Flip-flop (FF)	A single-bit clocked storage element used to register signals.
GHDL	An open-source simulator for VHDL designs.
Golden model	A trusted software reference implementation used to check hardware outputs.
Latency	The delay between data entering and the corresponding result leaving a circuit.
Line buffer	On-chip memory storing one image row to provide vertically aligned pixels for windowing.
LUT	Look-up table; the basic combinational logic element of an FPGA.
Pipelining	Partitioning a datapath into registered stages so multiple operations proceed concurrently.
PSNR	Peak signal-to-noise ratio; a logarithmic measure of image error, infinite for identical images.
RTL	Register-transfer level; a hardware description abstraction in terms of registers and logic.
Sobel operator	A pair of 3×3 kernels estimating horizontal and vertical image gradients.
Throughput	The rate at which results are produced, here in output pixels per second.
Timing closure	The state in which a synthesised circuit meets its target clock-period constraints.
WNS	Worst negative slack; the most negative timing margin reported by static timing analysis.

List of Symbols

Symbol	Description	Units
W	Image width	pixels
H	Image height	pixels
F	Frame rate	frames/s
Npx	Pixels per frame	pixels
Rpx	Required sustained pixel rate	pixels/s
fclk	Clock frequency	Hz
CPP	Clock cycles per output pixel	cycles
Tseq	Sequential-architecture throughput	pixels/s
Tpipe	Pipelined-architecture throughput	pixels/s
L	Total initial latency	cycles
Dw	Window-formation delay	cycles
Da	Arithmetic pipeline depth	cycles
Gx, Gy	Horizontal and vertical Sobel gradients	—
M	Approximate edge magnitude	—
p _{ij}	Pixel intensity at window row i, column j	—
S	Total weighted decision-matrix score	—
w _i , r _i	Decision-matrix weight and score for criterion i	—

1.0 Introduction

Embedded vision systems are deployed in robotics, automotive driver assistance, industrial inspection, and consumer devices, where they must analyse a continuous stream of camera images in real time. A common early step in such systems is edge detection, which identifies the boundaries between regions of an image and reduces the data that later stages must process. The Sobel operator is a widely used edge detector based on two-dimensional (2D) convolution with a small fixed kernel.

The engineering problem addressed in this report is that general-purpose processors may not guarantee the deterministic throughput and bounded latency required to apply Sobel filtering to a video stream in real time. A dedicated hardware accelerator described at the register-transfer level (RTL) and implemented on a field-programmable gate array (FPGA) can provide deterministic, high-throughput processing, but the designer must choose an appropriate architecture.

This report determines the most suitable RTL architecture for a real-time 3×3 Sobel accelerator. Section 2.0 provides background on 2D convolution, Sobel filtering, and hardware acceleration. Section 3.0 states the objective, aims, and limitations. Section 4.0 describes two candidate architectures, and Section 5.0 presents the initial assessment used to select between them. Section 6.0 details the proposed pipelined solution, Section 7.0 presents the engineering analysis together with the simulation results, and Section 8.0 discusses the findings. Sections 9.0 and 10.0 give the conclusions and recommendations.

2.0 Background

Two-dimensional convolution slides a small coefficient matrix, called a kernel, across an image and computes each output pixel as the weighted sum of a local neighbourhood of input pixels. The same multiply-accumulate pattern repeats for every pixel, making the operation computationally regular but data-intensive, and therefore well suited to hardware acceleration. The kernel coefficients determine the filtering behaviour, including blurring, sharpening, and gradient estimation [1].

Sobel edge detection applies two 3×3 kernels to estimate the horizontal and vertical intensity gradients, then combines them into an edge-magnitude image. It is favoured in embedded vision because it is computationally inexpensive, provides reasonable noise immunity through a built-in smoothing term, and maps efficiently onto fixed-function arithmetic; its small fixed kernel also makes it a representative case for studying convolution accelerators in general [2].

Real-time processing imposes a hard timing constraint: every pixel of every frame must be processed within a fixed budget. A central processing unit (CPU) executes convolution as instructions whose timing is affected by memory access and cache behaviour, introducing variability that can break the timing budget [3]. FPGAs and RTL architectures instead implement

the convolution as dedicated logic, forming a spatial datapath that processes pixels deterministically as they stream through, trading silicon area for throughput [4].

Four quantities dominate such designs. Throughput measures output pixels per unit time and determines whether the frame rate can be sustained. Latency measures the delay from input to output and matters for closed-loop systems. Resource usage measures the look-up tables (LUTs), flip-flops (FFs), digital signal processing (DSP) blocks, and block RAM (BRAM) consumed, constraining cost. Timing closure determines whether the circuit can meet its target clock period; a design that cannot close timing cannot run at the intended frequency regardless of its theoretical merit.

3.0 Objective

3.1 Problem

Real-time Sobel edge detection requires deterministic, sustained processing of a continuous pixel stream. A processor-based implementation may not provide guaranteed throughput and bounded latency at standard video resolutions and frame rates because instruction execution and memory access introduce variability. A dedicated RTL accelerator is required, but the most suitable architecture is not obvious.

3.2 Purpose

The purpose of this report is to determine the most suitable RTL hardware architecture for a real-time 3×3 Sobel edge-detection accelerator processing a streaming grayscale image source.

3.3 Aims

The aims of the project are to:

1. Support at least 640×480 grayscale video at 30 frames per second, a sustained rate of 9.216 million pixels per second.
2. Compare a sequential and a pipelined RTL architecture using cycles per output pixel as the primary throughput metric.
3. Estimate or measure FPGA resource usage (LUTs, flip-flops, DSP blocks, BRAM) using Xilinx Vivado synthesis [6].
4. Determine whether the selected design can meet a clock frequency sufficient for real-time processing of the target stream.
5. Verify the RTL output against a software reference (golden) model to a defined accuracy.

3.4 Limitations

- Only grayscale (single-channel) images are processed; RGB colour is excluded.
- Only the standard 3×3 Sobel operator is considered.

- No physical camera interface is required; inputs are stored images or test vectors.
- No full ASIC layout is performed and no neural-network inference is included.
- The implementation is verified in simulation using stored inputs rather than live video.

4.0 Potential Solutions

4.1 Sequential RTL Filter

The sequential RTL filter assembles a 3×3 pixel window and performs the Sobel arithmetic over several clock cycles using a small number of shared arithmetic units sequenced by a finite-state machine. Because few operations occur in parallel, it uses modest logic and is straightforward to design and verify. Its limitation is throughput: each output pixel requires multiple clock cycles, so the sustained rate is the clock frequency divided by the cycles per pixel, which may be insufficient at higher resolutions or frame rates.

4.2 Pipelined RTL Filter

The pipelined RTL filter uses two line buffers and shift registers to continuously assemble a 3×3 window from the incoming stream, and partitions the Sobel arithmetic into registered stages so that, after an initial fill delay, one output pixel is produced per clock cycle [4]. It is more complex and consumes more memory and registers, but it directly targets the high-throughput, low-latency streaming nature of the problem.

5.0 Initial Assessment

The two architectures are compared using a weighted decision matrix. Seven criteria, mapped onto the project aims and the engineering quantities of Section 2.0, are each weighted by importance and scored from 1 (poor) to 5 (excellent). The total weighted score for each candidate is given by Equation (1).

$$S = \sum (w_i \times r_i) \quad (1)$$

In Equation (1), S is the total weighted score, w_i is the weight of criterion i , and r_i is the score awarded to the candidate for criterion i , summed over all seven criteria. Throughput and latency receive the highest weights because the central problem is sustaining a deterministic real-time pixel rate; timing feasibility is weighted highly because a design that cannot close timing cannot meet the rate. Resource usage carries moderate weight, and implementation and verification complexity carry lower weight because they affect development effort rather than delivered performance.

Table 1: Weighted decision matrix for selecting the RTL Sobel architecture.

Criterion	Weight	Seq. r	Seq. w×r	Pipe. r	Pipe. w×r
Throughput	0.25	2	0.50	5	1.25

Latency	0.15	3	0.45	4	0.60
Hardware resource usage	0.15	5	0.75	3	0.45
Timing feasibility	0.15	4	0.60	4	0.60
Implementation complexity	0.10	5	0.50	2	0.20
Verification complexity	0.10	4	0.40	3	0.30
Relevance to industry skills	0.10	3	0.30	5	0.50
Total weighted score (S)	1.00		3.50		3.90

As shown in Table 1, the pipelined RTL filter achieves the higher total weighted score, driven by its superior throughput. The sequential design scores well on resource usage and simplicity but is penalised on throughput, the criterion most directly tied to the engineering problem. The pipelined RTL filter was therefore selected as the proposed solution, and the engineering analysis of Section 7.0 tests that selection quantitatively.

6.0 Proposed Solution

The proposed solution is a pipelined RTL Sobel edge-detection accelerator that processes a continuous stream of 8-bit grayscale pixels and produces one filtered output pixel per clock cycle in steady state. The design is written in VHDL and is organised into the hierarchy shown in Figure 1: a window-forming stage built from two line buffers and a bank of shift registers, followed by a two-stage arithmetic pipeline. Every stage is registered, so a new pixel is accepted on each clock edge once the pipeline has filled [4].

Incoming pixels enter one per clock cycle. The two line buffers each store one full image row, holding the two previous rows so that the three vertically aligned pixels of the current window column are available at the same time. Three 3-tap shift registers then provide the horizontal taps, presenting all nine pixels of the window (p_{00} through p_{22}) to the arithmetic stages in parallel. The window labelling and the two Sobel kernels are shown in Figure 2 [1].

The arithmetic datapath computes the horizontal gradient G_x and the vertical gradient G_y , forms the approximate magnitude $M \approx |G_x| + |G_y|$, and saturates the result to the 8-bit output range. A valid flag travels with the data through every pipeline register, so outputs produced while the buffers are filling, and along the image borders where no complete 3×3 window exists, are marked invalid; the output frame is therefore $(W - 2) \times (H - 2)$ pixels. Figure 3 shows the resulting behaviour: a one-time fill latency followed by one output pixel per clock cycle.

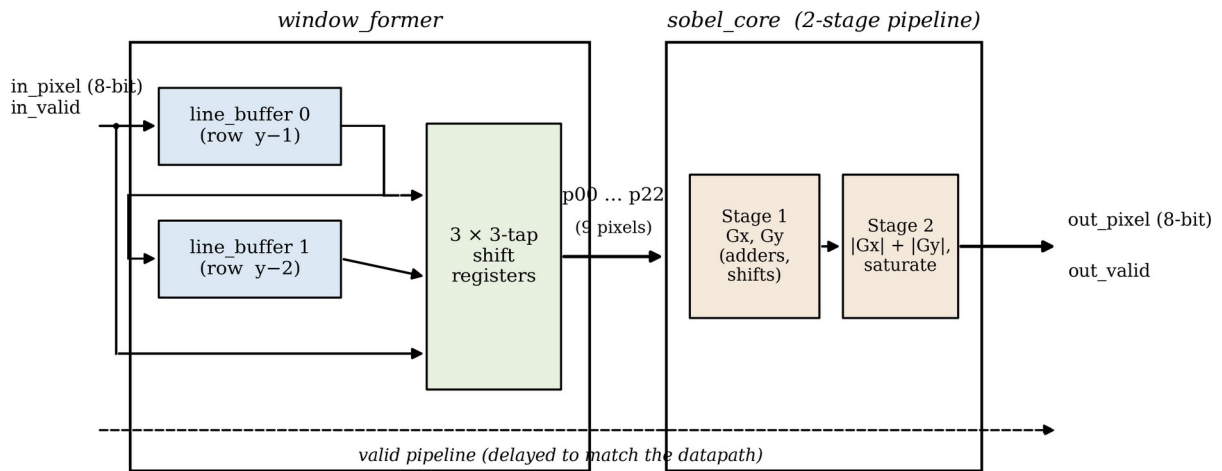


Figure 1: Block diagram of the pipelined Sobel accelerator.

3×3 pixel window			Gx kernel (horizontal)			Gy kernel (vertical)		
p00	p01	p02	-1	0	+1	-1	-2	-1
p10	p11	p12	-2	0	+2	0	0	0
p20	p21	p22	-1	0	+1	+1	+2	+1

Figure 2: The 3×3 pixel window and the horizontal (Gx) and vertical (Gy) Sobel kernels.

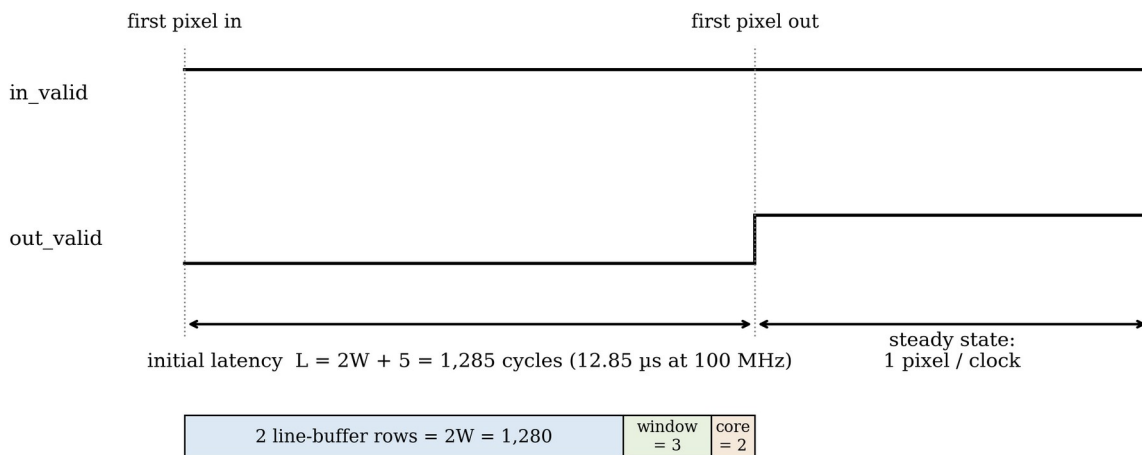


Figure 3: Pipeline timing, showing the fill latency and the one-pixel-per-cycle steady state.

7.0 Engineering Analysis

The analysis proceeds from the application requirement to the measured behaviour of the design. Section 7.1 establishes the pixel rate the accelerator must sustain, Sections 7.2 and 7.3 compare the throughput of the sequential and pipelined architectures, Section 7.4 derives and then measures the pipeline latency, and Section 7.5 examines the Sobel arithmetic and its bit widths. Sections 7.6 through 7.8 cover resource usage, verification against a software golden model, and the sensitivity and limitations of the analysis. All simulation results were obtained with the GHDL simulator (VHDL-2008) using a file-driven testbench and a 100 MHz testbench clock [5].

7.1 Real-Time Throughput Requirement

The first requirement is the sustained pixel rate that the accelerator must support. The number of pixels in one frame is the product of the image width and height, as given in Equation (2).

$$N_{px} = W \times H \quad (2)$$

In Equation (2), N_{px} is the number of pixels in one frame, W is the image width in pixels, and H is the image height in pixels. The required sustained pixel rate is the pixels per frame multiplied by the frame rate, as in Equation (3).

$$R_{px} = W \times H \times F \quad (3)$$

In Equation (3), R_{px} is the required pixel rate in pixels per second and F is the frame rate in frames per second. For the target of 640×480 grayscale video at 30 frames per second, Equation (3) gives a required rate of $640 \times 480 \times 30 = 9.216 \times 10^6$ pixels per second. Any candidate architecture must sustain at least this rate to meet the project aim.

7.2 Sequential RTL Throughput

A sequential architecture shares a small number of arithmetic units across the Sobel computation, so it needs several clock cycles for each output pixel. The exact count depends on how many multiply-accumulate and absolute-value steps are serialised. The sustained throughput of such a design is given by Equation (4).

$$T_{seq} = f_{clk} / CPP \quad (4)$$

In Equation (4), T_{seq} is the sequential throughput in output pixels per second, f_{clk} is the clock frequency in hertz, and CPP is the number of clock cycles required per output pixel. At $f_{clk} = 100$ MHz and a representative $CPP = 8$, the sequential throughput is 1.25×10^7 pixels per second. This nominally exceeds the 640×480 target, but the margin shrinks quickly as resolution increases or as CPP grows with control overhead, and it leaves little headroom for higher frame rates.

7.3 Pipelined RTL Throughput

In the pipelined architecture every stage is registered and operates concurrently. Once the line buffers and pipeline registers have filled, the design accepts one input pixel and produces one output pixel on every clock cycle, so the steady-state throughput reduces to Equation (5).

$$T_{pipe} = f_{clk} \quad (5)$$

In Equation (5), T_{pipe} is the pipelined steady-state throughput in output pixels per second and f_{clk} is the clock frequency in hertz. At 100 MHz the design sustains 1.0×10^8 pixels per second, roughly ten times the requirement from Section 7.1 and an order of magnitude above the sequential estimate of Section 7.2.

Simulation confirmed this behaviour. In the 640×480 run the testbench streamed all 307,200 input pixels through the design with no stalls, one pixel per clock, and the full frame completed in 3.085 ms of simulated time. At 100 MHz this corresponds to approximately 324 frames per second, so the measured steady-state throughput matches Equation (5).

7.4 Latency Analysis

Latency is the delay between a pixel entering the accelerator and the corresponding output pixel becoming valid, and it is distinct from throughput. For this design the latency has three contributions: the line-buffer fill, the window-formation delay, and the arithmetic pipeline depth, as expressed in Equation (6).

$$L = 2W + D_w + D_a \quad (6)$$

In Equation (6), L is the total latency in clock cycles, W is the image width in pixels, D_w is the window-formation delay of the shift-register stage in cycles, and D_a is the depth of the arithmetic pipeline in cycles. Two full image rows ($2W = 1,280$ cycles at $W = 640$) must enter the line buffers before the first complete window column exists; window formation adds $D_w = 3$ cycles and the two registered arithmetic stages add $D_a = 2$ cycles, for a predicted latency of 1,285 cycles.

The measured latency in simulation was exactly 1,285 clock cycles (12.85 μ s at 100 MHz) from the first pixel clocked in to the first assertion of the output valid signal, which confirms the model in Equation (6). This latency is paid once at the start of the stream and does not reduce steady-state throughput; against a 33 ms frame period at 30 frames per second it is negligible.

7.5 Sobel Arithmetic Analysis

The Sobel operator estimates image gradients with two 3×3 kernels applied to the window pixels p_{00} through p_{22} , where p_{00} is the top-left pixel and p_{22} is the bottom-right pixel (Figure 2). The horizontal gradient is computed using Equation (7).

$$G_x = (p_{02} + 2p_{12} + p_{22}) - (p_{00} + 2p_{10} + p_{20}) \quad (7)$$

In Equation (7), G_x is the horizontal gradient estimate and the p_{ij} terms are the 8-bit pixel intensities at row i , column j of the window. The vertical gradient is computed using Equation (8).

$$G_y = (p_{20} + 2p_{21} + p_{22}) - (p_{00} + 2p_{01} + p_{02}) \quad (8)$$

In Equation (8), G_y is the vertical gradient estimate, with the p_{ij} terms defined as above. The kernel coefficients are limited to -2 , -1 , 0 , 1 , and 2 . Multiplication by two is a hard-wired left shift and multiplication by one requires no logic, so both gradients are implemented with adders, subtractors, and shifts only, avoiding DSP-block usage and reducing logic depth [2]. The edge magnitude is approximated by Equation (9).

$$M \approx |G_x| + |G_y| \quad (9)$$

In Equation (9), M is the approximate edge-magnitude output and $|G_x|$ and $|G_y|$ are the absolute values of the two gradients. The approximation avoids the square root of the true Euclidean magnitude at the cost of a small, well-understood directional error [2]. Bit-width growth is bounded rather than assumed: the largest possible gradient magnitude is $4 \times 255 = 1,020$, which fits in a 13-bit signed word, and the sum $|G_x| + |G_y|$ is at most 2,040, held in 14 bits before the final saturation to the 8-bit output range. The implemented datapath uses exactly these widths, so no intermediate sum can overflow.

7.6 Resource Usage Analysis

FPGA resource usage will be finalised from Xilinx Vivado synthesis of the RTL [6]. The metrics of interest are look-up tables (LUTs), flip-flops, DSP blocks, and block RAM, together with the maximum clock frequency from static timing analysis. Table 2 lists the architectural estimates; each value follows directly from the structure of the design and will be replaced by the corresponding Vivado report figure once synthesis is run.

Table 2: Expected synthesis results (pre-synthesis architectural estimates, to be confirmed in Vivado).

Metric	Expected value	Basis of estimate
Look-up tables (LUTs)	$\approx 300\text{--}600$	adder/subtractor trees and control logic
Flip-flops (FFs)	$\approx 400\text{--}700$	window shift registers and pipeline registers
DSP blocks	0	no multipliers; coefficients use shifts and adds
Block RAM	$2 \times 18 \text{ Kb}$	one 640×8 -bit row buffer per line buffer (5,120 bits each)
Maximum clock frequency	$> 100 \text{ MHz}$	shallow logic between registers (2-stage arithmetic)

The two line buffers dominate memory use and are written in the read-first style that Vivado infers as block RAM [6]. Because the datapath contains no multipliers, DSP usage is expected to be zero by construction rather than by optimisation.

7.7 Verification Results

The RTL is verified by direct comparison against a software golden model that implements the same integer arithmetic in Python. A conversion script turns a source image into a text stream of pixel values, the VHDL testbench streams the pixels through the design and records the output stream, and the model then compares every output pixel [5]. Because the RTL and the golden model use identical integer arithmetic, any mismatch indicates a real design fault rather than rounding noise.

Four complete runs were performed at increasing frame sizes, including a synthetic directed-edge image and a full 640×480 frame. The results are summarised in Table 3. In all four runs every output pixel matched the golden model exactly: 100.0000 % exact match, zero maximum absolute error, and therefore infinite peak signal-to-noise ratio (PSNR).

Table 3: Verification results, RTL output versus software golden model (GHDL simulation).

Input frame	Output frame	Output pixels	Exact match	Max abs error	PSNR
32 × 32	30 × 30	900	100.0000 %	0	∞ dB
48 × 48	46 × 46	2,116	100.0000 %	0	∞ dB
256 × 256	254 × 254	64,516	100.0000 %	0	∞ dB
640 × 480	638 × 478	304,964	100.0000 %	0	∞ dB

Figure 4 shows the 640×480 case: the input image, the RTL output, the golden-model output, and the pixel-by-pixel difference image, which is uniformly zero. Reset and border behaviour are exercised by every run, since each simulation starts from reset and the valid logic must suppress the border pixels for which no complete 3×3 window exists; the output frame size of $(W - 2) \times (H - 2)$ pixels was produced exactly in all four runs.

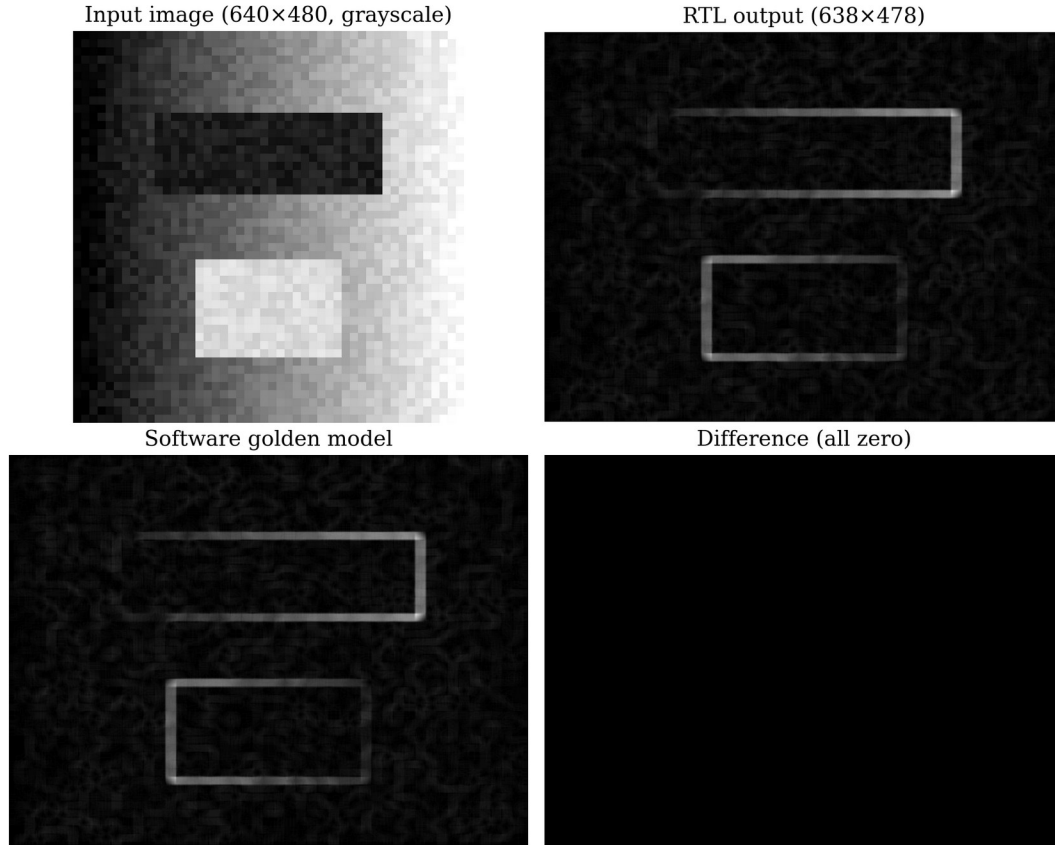


Figure 4: Verification of the 640×480 run: input, RTL output, golden model, and difference image.

7.8 Sensitivity and Limitations of the Analysis

The real-time margin is sensitive to three parameters. From Equations (3) and (5), the margin is the ratio of f_{clk} to $W \times H \times F$: raising the resolution or frame rate increases the requirement linearly, while a lower achievable clock frequency reduces the supply. The sequential alternative also scales inversely with CPP from Equation (4), which makes it far more sensitive to control overhead; the pipelined design keeps a large margin across plausible operating points.

The analysis itself has limits that should be kept in view. It covers grayscale images and the fixed 3×3 Sobel operator only; larger kernels would add line buffers and arithmetic depth, and RGB processing would widen or multiplex the datapath. The throughput and latency figures assume an ideal stall-free input stream, so a real camera or bus interface with back-pressure would need the ready/valid handshake noted in the proposed solution. The 100 MHz clock is a testbench assumption until Vivado timing analysis confirms it, and the resource figures in Table 2 remain estimates until synthesis. Finally, because the golden model shares its integer arithmetic with the RTL, the verification establishes that the hardware implements the intended computation; it does not evaluate how well the Sobel operator itself detects edges, which is a property of the algorithm rather than of this design.

8.0 Discussion

The measurements support the choice of the pipelined architecture. The design sustains one output pixel per clock cycle in simulation, which at 100 MHz is about ten times the 9.216×10^6 pixels-per-second requirement derived in Section 7.1 and an order of magnitude beyond the sequential estimate. Throughput is the quantity most directly tied to the engineering problem of deterministic real-time processing, so this margin is the decisive result.

The cost of that margin is modest. The extra hardware is dominated by the two line buffers, which fit in two 18 Kb block RAMs, and by pipeline registers; the arithmetic uses no multipliers at all. The design therefore trades a small, cheap amount of memory for a tenfold throughput margin, which is a favourable exchange on any modern FPGA.

The exact agreement between the predicted and measured latency (1,285 cycles in both cases) is worth emphasising, because it validates the analytical model of Equation (6) rather than just the simulation. Latency and throughput remain distinct quantities: the 12.85 μ s fill delay is paid once per stream and is negligible against a 33 ms frame period, although a latency-critical closed-loop application would still need to budget for it explicitly.

Two results remain provisional. Timing closure has not yet been demonstrated: the throughput of Equation (5) holds only if the synthesised design meets the 100 MHz clock, and although the shallow two-stage arithmetic pipeline makes this likely, the claim is confirmed only by the Vivado timing report. Likewise the resource figures in Table 2 are architectural estimates until the synthesis utilisation report replaces them. Verification confidence is bounded by the test set: four image sizes provide strong evidence of correctness, but they do not constitute a formal proof.

Beyond the immediate project, the results generalise in useful ways. The measured 100 Mpixel/s capability already exceeds the 62.2×10^6 pixels-per-second demand of 1080p video at 30 frames per second, so the same architecture would support full-HD edge detection with no structural change beyond longer line buffers. The line-buffer-and-window pattern is not specific to Sobel: any 3 $\times$ 3 kernel, including Gaussian smoothing or sharpening, and the first convolutional layers of neural networks reuse the same streaming structure, so both the design and the analysis method transfer directly to other streaming vision accelerators. The natural next steps are to run Vivado synthesis and timing analysis to replace the estimated figures, to capture simulation waveforms for documentation, and to add an AXI-Stream wrapper so the block can be dropped into a larger video pipeline.

9.0 Conclusions

The pipelined RTL filter is the most suitable architecture for the real-time Sobel accelerator, and the purpose of the report has been met. The primary aim is satisfied with a wide margin: the design sustained one output pixel per clock cycle in simulation, equivalent to 100 million pixels

per second at a 100 MHz clock against a requirement of 9.216 million pixels per second, and a full 640×480 frame completes in 3.085 ms, or about 324 frames per second.

The design is functionally correct with respect to its reference model. Across four verification runs, up to and including the full 640×480 frame, every output pixel matched the software golden model exactly. The measured pipeline latency of 1,285 clock cycles agreed exactly with the analytical prediction, validating the latency model as well as the implementation.

The pipelined design outperforms the sequential alternative by roughly an order of magnitude on the cycles-per-pixel metric, and its arithmetic is implemented entirely with adders, subtractors, and shifts, so the resource cost is dominated by two small row buffers of block RAM rather than scarce DSP blocks. The conclusions on FPGA resource usage and achievable clock frequency remain provisional until Vivado synthesis and timing reports replace the estimates of Table 2, and all conclusions hold within the stated limitations of grayscale 3×3 filtering and an idealised stall-free pixel stream.

10.0 Recommendations

1. Run Vivado synthesis and implementation, replace the estimates in Table 2 with the reported values, and confirm timing closure at 100 MHz.
2. Capture annotated simulation waveforms of the window taps, out_valid, and out_pixel signals and include them in Appendix B.
3. Add an AXI-Stream ready/valid wrapper so the accelerator interoperates with back-pressured streaming systems.
4. Extend the design to RGB images and configurable kernel coefficients to broaden its applicability.
5. Benchmark the accelerator against CPU and high-level-synthesis implementations to quantify the benefit.
6. Strengthen verification with randomised input images and assertion-based checks on the valid-signal timing.

References

- [1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. New York, NY: Pearson, 2018.
- [2] N. Kanopoulos, N. Vasanthavada, and R. L. Baker, "Design of an image edge detection filter using the Sobel operator," *IEEE J. Solid-State Circuits*, Vol. 23, No. 2, pp. 358–367, Apr. 1988.
- [3] B. Kisačanin, S. S. Bhattacharyya, and S. Chai, Eds., *Embedded Computer Vision*. London, U.K.: Springer-Verlag, 2009.

- [4] D. G. Bailey, *Design for Embedded Image Processing on FPGAs*. Singapore: John Wiley & Sons (Asia), 2011.
- [5] T. Gingold, "GHDL Documentation," [Online]. [Cited 2026 July 3]. Available at: <https://ghdl.github.io/ghdl/>
- [6] *Vivado Design Suite User Guide: Synthesis*, UG901 (v2024.1), Advanced Micro Devices, Inc., San Jose, CA, 2024.

Appendix A: Supporting Calculations

This appendix presents the worked calculations summarised in Section 7.0. All figures use the project parameters $W = 640$, $H = 480$, $F = 30$ frames per second, and $f_{clk} = 100$ MHz.

A.1 Required Pixel Rate

$$R_{px} = W \times H \times F = 640 \times 480 \times 30 = 9,216,000 \text{ pixels per second.}$$

A.2 Sequential Throughput Estimate

$$T_{seq} = f_{clk} / CPP = 100 \times 10^6 / 8 = 12.5 \times 10^6 \text{ pixels per second, a margin of only } 12.5 / 9.216 \approx 1.36 \text{ over the requirement.}$$

A.3 Pipelined Throughput and Margin

$T_{pipe} = f_{clk} = 100 \times 10^6$ pixels per second, a margin of $100 / 9.216 \approx 10.9$ over the requirement. For 1080p at 30 frames per second the demand is $1920 \times 1080 \times 30 = 62.2 \times 10^6$ pixels per second, still within the measured capability.

A.4 Latency

Predicted: $L = 2W + D_w + D_a = 1,280 + 3 + 2 = 1,285$ cycles = $12.85 \mu\text{s}$ at 10 ns per cycle.

Measured: the first output-valid assertion occurred 12,850 ns after the first pixel was clocked in, which is 1,285 cycles, matching the prediction exactly.

A.5 Frame Time and Frame Rate

A full frame requires the 307,200 input pixels plus the fill latency: $(307,200 + 1,285) \times 10 \text{ ns} \approx 3.085 \text{ ms}$, giving $1 / 3.085 \text{ ms} \approx 324$ frames per second. The simulated end-of-run time for the full frame was 3,085,165 ns, consistent with this figure.

A.6 Bit-Width Bounds

The largest weighted sum in either gradient is $(255 + 2 \times 255 + 255) = 1,020$, so G_x and G_y lie in $[-1,020, 1,020]$ and fit in 13-bit signed words. The magnitude sum $|G_x| + |G_y|$ is at most 2,040, held in 14 unsigned bits before saturation to the 8-bit output range.

A.7 Line-Buffer Memory

Each line buffer stores one image row of $W \times 8 \text{ bits} = 640 \times 8 = 5,120$ bits, which fits in a single 18 Kb block RAM; the design therefore needs two block RAMs in total.

Appendix B: Simulation Waveforms and Synthesis Reports

This appendix is reserved for the Vivado artefacts that accompany the final submission: annotated XSim waveform captures showing the window taps, the out_valid assertion at the end of the fill latency, and the steady-state one-pixel-per-cycle output; the post-synthesis utilisation report (LUTs, flip-flops, DSP blocks, and block RAM); and the timing summary confirming the worst negative slack at the 100 MHz constraint. These reports replace the estimates given in Table 2.

[Insert Vivado waveform captures and the utilisation and timing reports here once synthesis is complete.]